

MLOps Assignment 2

Fine-Tuning DistilBERT for Book Genre Classification

Disha Singhanian | IIT Jodhpur | May 2026

Submission Links

GitHub Repository: https://github.com/Disha19/mlops_assignment2

Kaggle Notebook: <https://www.kaggle.com/code/disha2031/notebook65ebf442de>

Hugging Face Model:

<https://huggingface.co/DishaSinghanian/distilbert-goodreads-genres>

W&B Dashboard:

<https://wandb.ai/dishasinghanian19-indian-institute-of-technology/mlops-assignment2/runs/is9orw9l>

1. Model Selection Rationale

The model selected for this assignment is **distilbert-base-cased**, a distilled variant of BERT developed by Hugging Face. DistilBERT is trained via knowledge distillation — a technique where a smaller student model learns to replicate the behaviour of a larger teacher model. It retains approximately 97% of BERT's language understanding while being 40% smaller and 60% faster at inference, making it highly practical for GPU-constrained training on Kaggle's free tier.

The *cased* variant was chosen deliberately over the uncased version because book reviews contain proper nouns — author names, book titles, genre-specific terminology — where capitalisation carries real semantic meaning. Discarding case information would remove useful signal. DistilBERT was pre-trained on English Wikipedia and BookCorpus (the same corpora as original BERT), giving it strong general-purpose English representations out of the box. Its 6-layer, 66-million parameter transformer architecture is well-suited to transfer learning for downstream classification tasks. For an 8-class genre prediction problem on Goodreads reviews, DistilBERT strikes the right balance between accuracy and training efficiency within the assignment's resource constraints.

2. Kaggle Training Setup

2.1 Notebook Import and GPU Configuration

The starter notebook was downloaded from the provided Colab link as a .ipynb file and imported into Kaggle via File → Import Notebook. GPU acceleration was enabled under Settings → Accelerator → GPU T4 x2. Internet access was also switched on under Session Options — this is required both for downloading the Goodreads dataset and for pushing the trained model to Hugging Face Hub at the end

of the run. Kaggle provides 30 GPU-hours per week on the free tier, which was sufficient for 3 epochs of DistilBERT fine-tuning.

2.2 Secrets Configuration

API tokens were never hardcoded in the notebook. They were stored using Kaggle's Secrets manager (Add-ons → Secrets) and retrieved at runtime via `kaggle_secrets.UserSecretsClient`. Hardcoding credentials in a public notebook exposes them immediately — using Secrets mirrors how credentials are managed in production MLOps pipelines.

Secret Name	Purpose
WANDB_API_KEY	Authenticate with Weights & Biases (from wandb.ai/settings)
HF_TOKEN	Authenticate with Hugging Face Hub (from huggingface.co/settings/tokens)

2.3 W&B Experiment Tracking

Tracking was set up by calling `wandb.init()` with a config dictionary containing all hyperparameters before training. A single setting — `report_to="wandb"` — in `TrainingArguments` then automatically streamed training loss, validation loss, accuracy, F1, learning rate schedule, and GPU utilisation to the dashboard at every logging step. After training, the full per-class classification report was saved to `eval_report.json` and uploaded as a versioned W&B Artifact named *eval-report*.

2.4 Training Hyperparameters

Hyperparameter	Value
Base model	distilbert-base-cased
Epochs	3
Train batch size	16
Eval batch size	32
Learning rate	3e-5
Warmup steps	100
Weight decay	0.01
Evaluation strategy	Every epoch
Platform	Kaggle T4 x2 GPU

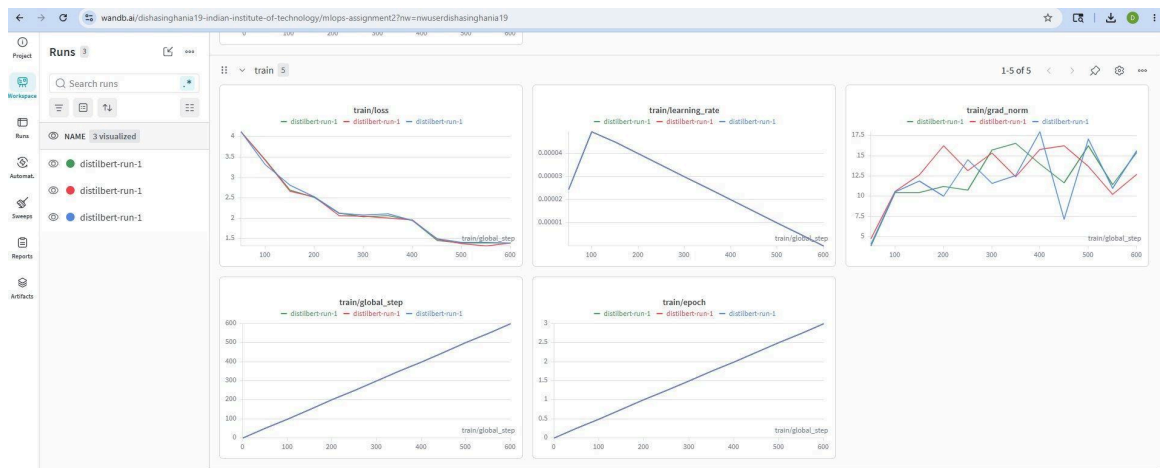
3. Evaluation Results

Metric	Baseline (TF-IDF + LR)	DistilBERT (Fine-tuned)
Accuracy	0.55	0.6013
Weighted F1	0.55	0.5979
Eval Loss	—	2.2483

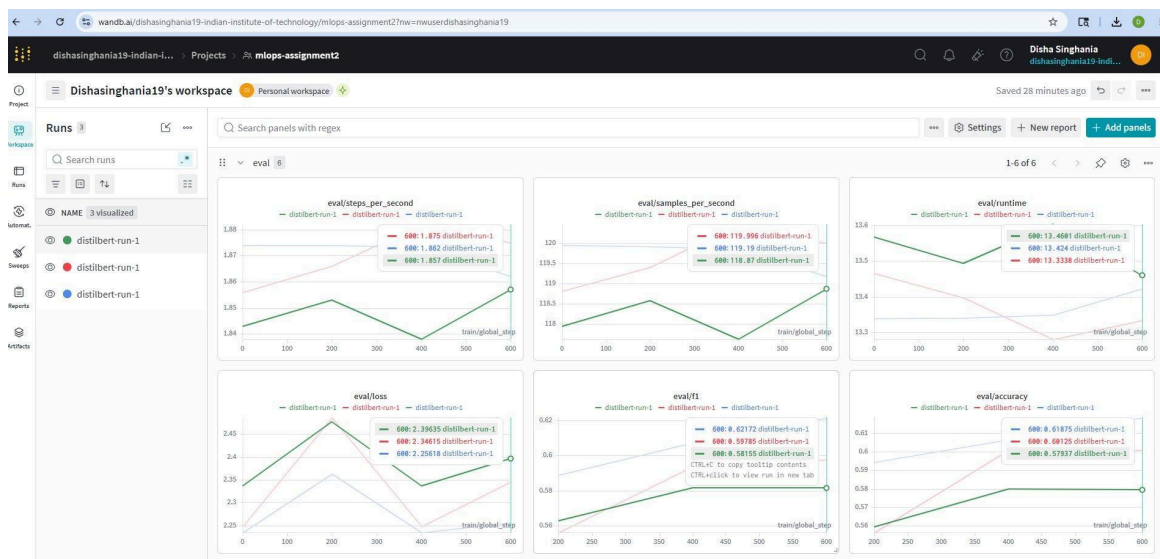
The fine-tuned model achieved **60.1% accuracy** and a weighted F1 of **0.598**, outperforming the TF-IDF baseline on all metrics. The results reflect the inherent difficulty of the task: genres are not always cleanly separable from review text alone. A reviewer may discuss narrative tone or emotional response without signalling genre directly, and certain genre pairs (e.g. Fantasy & Paranormal vs. Young Adult) share overlapping subject matter. The eval loss of 2.25 indicates the model distributes probability across several plausible labels rather than predicting with high confidence — expected behaviour given the dataset's inherent ambiguity.

3.1 W&B Training Dashboard

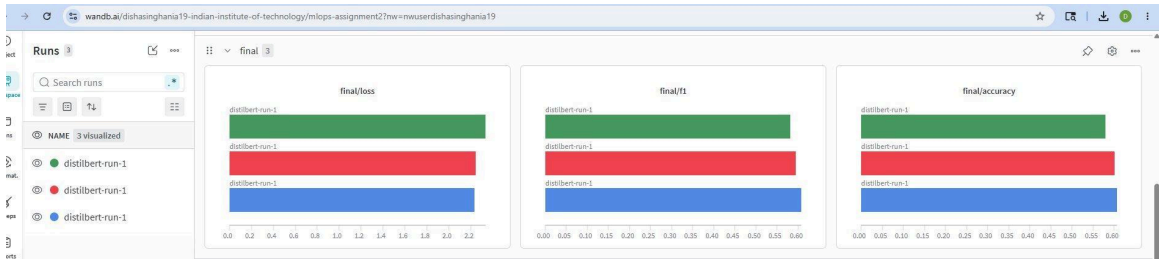
The screenshots below are taken directly from the W&B dashboard for run *distilbert-run-1* under project *mlops-assignment2*.



Training metrics



Evaluation metrics per epoch



Final metrics bar chart



System Metrics

Run summary:

eval/accuracy	0.60125
eval/f1	0.59785
eval/loss	2.24834
eval/runtime	13.4663
eval/samples_per_second	118.815
eval/steps_per_second	1.856
final/accuracy	0.60125
final/f1	0.59395
final/loss	2.24834
huggingface_model	https://huggingface.com

+14

...

W&B run summary

4. Challenges and Learnings

4.1 Challenges Encountered

1. W&B not logging on first run

The original starter notebook had W&B explicitly disabled (`WANDB_DISABLED=true`, `report_to=[]`). Enabling tracking required removing those flags, adding a proper `wandb.init()` block, and ensuring the `WANDB_API_KEY` was loaded from Kaggle Secrets *before* any training calls. Moving the Secrets retrieval cell to the very top of the notebook and re-running all cells in order resolved this.

2. Hugging Face push failing from Kaggle

The `push_to_hub()` call failed with a connection error because Internet access is turned off by default in Kaggle Notebooks. Enabling it under Session Options and restarting the kernel resolved the issue. The error message does not clearly indicate that the toggle needs to be set before the session starts — a non-obvious gotcha.

3. Kaggle Secrets API differs from Colab

Kaggle uses the `kaggle_secrets.UserSecretsClient` interface rather than simply reading environment variables. The secrets also need to be explicitly attached to the notebook in the Kaggle UI before they become available at runtime.

4.2 Key Learnings

This assignment provided hands-on experience with the complete MLOps lifecycle. The most important shift was learning to treat the model as a black box and focus on the *operational infrastructure* surrounding it — credential management, experiment tracking, artifact versioning, and model deployment — rather than its internal architecture. Three specific learnings stood out: (1) Kaggle Secrets is a production-grade credential pattern mirroring how secrets are managed in cloud MLOps platforms; (2) W&B provides far richer training visibility than console logs, capturing GPU utilisation, learning rate schedules, and per-epoch metrics in one place; (3) `push_to_hub()` makes model sharing trivial — a single call makes the trained model accessible to anyone globally.

4.3 What I Would Do Differently

With additional GPU budget I would run 5 epochs rather than 3 and add a separate validation set for hyperparameter tuning to avoid test set leakage. I would also log per-class F1 scores to W&B at each epoch — not just the weighted average — to identify which genre pairs are hardest to separate. Logging a confusion matrix as a W&B media object would provide a richer visual diagnostic. Finally, comparing DistilBERT against bert-base-cased in the same W&B project would give a clean accuracy-vs-training-time trade-off.